

Quant AI Platform

Multi-Horizon Forecasting and Real-Time Market Intelligence

Prepared By: Vinay Yadav

1 Executive Summary & Technology Stack

The Quant AI Platform is a fully autonomous, end-to-end algorithmic trading and market analysis engine. Designed to process continuous multimodal data streams, the system leverages advanced machine learning to generate multi-horizon equity forecasts (1-day, 3-day, and 5-day trajectories). Moving beyond localized model training, this project encompasses a full-scale MLOps and cloud infrastructure pipeline, integrating real-time data ingestion, natural language processing, and automated trade signal generation.

Core Technology Stack

- **Machine Learning:** LightGBM, Scikit-Learn, FinBERT, Hugging Face Transformers, MLflow.
- **Data Engineering:** Apache Kafka, PostgreSQL (AWS RDS), Pandas, NumPy.
- **Backend Infrastructure:** FastAPI, Python, Docker, Google Cloud Platform (GCP) Compute Engine.
- **Frontend & Deployment:** Next.js (React), Tailwind CSS, Framer Motion, Vercel.

2 Aim and Problem Statement

2.1 Problem Statement

Traditional quantitative models rely heavily on historical price action, often failing to capture sudden market regime changes triggered by macroeconomic news or shifts in retail/institutional sentiment. Furthermore, deep learning architectures frequently overfit the highly noisy, non-stationary nature of financial time-series data.

2.2 Objective

The primary objective of this project is to build an institutional-grade pipeline that:

1. Fuses traditional technical/statistical market data with real-time NLP sentiment analysis.
2. Deploys a robust, tree-based machine learning architecture capable of resisting overfitting.
3. Generates high-confidence, multi-hop forecasts (1-day, 3-day, 5-day) to execute Long/Short equity strategies.
4. Operates autonomously across a distributed cloud architecture (AWS for storage, GCP for compute/inference).

3 Data Engineering & Pipeline Integration

The predictive power of the engine relies on a multimodal data approach, combining structured numeric data with unstructured text data.

3.1 Market Data Collection

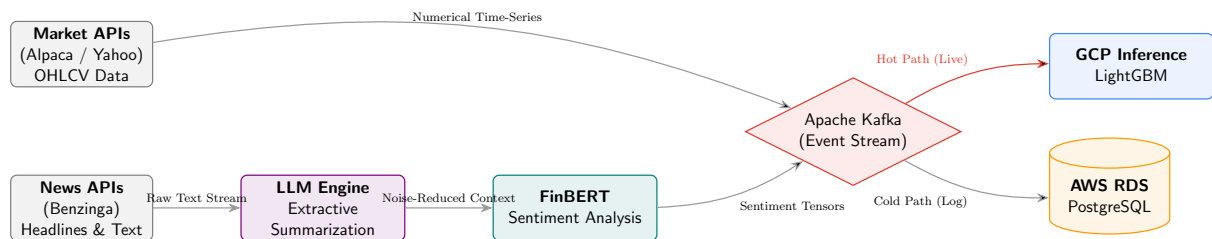
Historical and live OHLCV (Open, High, Low, Close, Volume) data is ingested via high-frequency financial APIs. This raw data forms the foundational time-series dataset upon which statistical and technical indicators are computed.

3.2 Alternative Data: News & NLP Pipeline

To capture market psychology, the pipeline continuously ingests financial news headlines and articles.

- **LLM Summarization:** Raw financial news is often dense. We utilize an LLM to perform extractive and abstractive summarization, isolating core catalysts.
- **FinBERT Sentiment Extraction:** The summarized text is passed through FinBERT, a pre-trained NLP model fine-tuned on financial corpora. FinBERT generates dense sentiment embeddings (Positive, Negative, Neutral probabilities) which are seamlessly joined with numerical market data.

Multimodal Data Pipeline Architecture (Streaming Topology)



4 Feature Engineering

The model evaluates a high-dimensional feature space of approximately 102 distinct features. Data is transformed into stationary, normalized inputs.

- **Price Action & Momentum:** Moving Averages (7d, 20d, 50d, 200d), MACD, RSI, and Stochastic Oscillators.
- **Volatility & Statistical:** Bollinger Bands, Average True Range (ATR), Rolling Standard Deviations, and Z-Scores.
- **NLP & Sentiment:** `sentiment_7d_avg`, `news_count_7d`, and Sentiment Momentum.
- **Macro & Cross-Asset:** Broad market correlation indices (e.g., `QQQ_return_5d`) and sector momentum.

5 Model Architecture & Evolution

5.1 The Failure of LSTM and Attention Mechanisms

Initial iterations utilized Long Short-Term Memory (LSTM) networks combined with Attention mechanisms. This architecture failed in production due to:

1. **Overfitting to Noise:** The deep capacity of the LSTM led to severe overfitting, mapping random market walks as deterministic patterns.
2. **Vanishing Gradients on Multimodal Data:** Balancing dense NLP embeddings with volatile price data required complex normalization that the Attention mechanism struggled to dynamically weight.
3. **Inference Latency:** Autoregressive decoding for multi-hop forecasts introduced unacceptable latency.

5.2 The Shift to LightGBM (MultiOutput)

The architecture was migrated to a **LightGBM** model encapsulated in a MultiOutput wrapper.

- Tree-based models are inherently better at handling tabular data with heterogeneous features (mixing technicals with NLP scores).
- The MultiOutput wrapper predicts 1-day, 3-day, and 5-day targets simultaneously, preserving the temporal structure without autoregressive lag.

5.3 Why LightGBM over XGBoost?

LightGBM was specifically chosen for this high-frequency pipeline because of its tree growth strategy:

- **Leaf-wise Growth:** LightGBM grows trees leaf-wise (best-first), minimizing loss efficiently, unlike XGBoost's memory-intensive level-wise growth.
- **Histogram-based Binning:** LightGBM bins continuous features into discrete buckets, drastically reducing memory usage and training time for our 102-feature dataset.

6 System Architecture & Cloud Integration

The platform operates as a distributed microservices architecture, spanning AWS and GCP to ensure high availability and stream processing capability.

6.1 AWS Services: Data Logging & Storage

- **PostgreSQL (AWS RDS):** The highly normalized relational storage for processed features, historical market data, and generated AI trade signals. It serves as the asynchronous "Cold Path" for the streaming architecture, optimized for rapid read queries from the frontend edge.

6.2 GCP Services: Compute, Kafka, & MLOps

The inference engine and backend API are hosted on Google Cloud Platform (GCP).

- **Apache Kafka (Streaming Hub):** Operates as the central nervous system. Data ingested from external APIs is pushed to Kafka topics. Kafka then distributes the data continuously via two routes: directly to the in-memory inference engine for real-time processing (Hot Path), and to the PostgreSQL database for historical logging (Cold Path).
- **Dockerized Backend:** The Python inference engine is containerized, holding the FastAPI application, LightGBM model binary, and data scaling logic.
- **MLflow Integration:** Tracks model metrics (Validation RMSE, L2 Loss) and handles model registry, allowing for automated continuous retraining without API downtime.

7 Real-Time Streaming Topology (Lifecycle Workflow)

To circumvent the disk I/O latency inherent in traditional database-first architectures, the platform utilizes a true real-time streaming topology.

7.1 The Active Pipeline Flow

1. **Data Collection & NLP Processing (GCP):** The ingestion scripts continuously poll financial and news APIs. Text data is immediately processed by the local LLM summarizer and FinBERT sentiment analyzer.
2. **Event Streaming (Kafka Producer):** The newly structured, highly dimensional data packet is published directly to the Apache Kafka cluster as an event stream.
3. **The Streaming Split (Hot vs. Cold):**
 - **Hot Path (Inference):** The GCP Docker container, acting as a Kafka Consumer, pulls the live data straight into memory. The LightGBM model processes the stream and generates immediate multi-horizon signals.
 - **Cold Path (Logging):** Simultaneously, a separate consumer writes the raw structured data to the AWS RDS PostgreSQL database for historical durability and subsequent model backtesting.
4. **Signal Storage (GCP → AWS):** Once the in-memory predictions are made, the model pushes the final trade signals directly to the AWS RDS database.
5. **Client Delivery (Vercel → AWS):** The Next.js frontend deployed on Vercel's edge network securely queries the AWS RDS database to populate the live interactive dashboard.

System Architecture Flowchart (Real-Time Streaming Topology)

